

开源鸿蒙移动应用创新成果申报

星门 StarGate

面向 HarmonyOS 的应用原子能力智能聚合与直达平台

跨过 App 的门，一步抵达功能

多应用覆盖 • 多场景分类 • 持续扩展

项目负责人：刘发荣

成果完成单位：软件学院 SMAT 项目组

2026 年 11 月

摘 要

移动应用已经覆盖生活与工作的几乎全部场景，但用户真正需要的往往不是某个 App，而是 App 里的某一个功能。功能被层层菜单包裹，用户需要先找 App、再找入口，路径长、记忆成本高；系统侧也只掌握“安装了哪些 App”，并不掌握“能直达哪些功能”。星门 StarGate 以 HarmonyOS NEXT 的桌面卡片为入口，提出“应用原子能力”的聚合与直达方法：从公开 Scheme、App Linking、系统 Shortcut 与人工复核中生产候选能力，经真机验证后写入统一能力目录，再通过桌面卡片、最近添加、A－Z# 分段搜索与分类浏览等入口呈现，用户点击图标即可落到目标功能页。系统采用五层解耦架构，核心逻辑与 UI、系统 API 分离，配套自动化测试与真机验证流程；能力库覆盖主流应用与多个生活分类，并可随安装状态与能力发现持续扩展。星门把“找 App”变成“找功能”，为新形态操作系统上的能力呈现提供了一种可落地、可解释、可扩展的范式。

关键词：HarmonyOS NEXT；原子能力；桌面卡片；能力呈现；推荐排序；应用直达

目 录

第 1 章 背景与意义 5
1.1 移动应用生态的“功能孤岛” 5
1.2 用户侧：找功能比找 App 更难 5
1.3 平台侧：HarmonyOS 卡片带来的机会 5
1.4 项目意义 6
1.5 项目目标与边界 6
第 2 章 产品定义与使用场景 6
2.1 一句话定义 6
2.2 目标用户 7
2.3 四个入口 7
2.4 典型场景 7
2.5 使用流程 7
2.6 设计原则 8
第 3 章 总体设计 8
3.1 设计目标 8
3.2 五层架构 8
3.3 能力从发现到呈现 9
3.4 关键数据结构 10
3.5 模块清单 11
第 4 章 路线探索与最终选择 11
4.1 四条路线 11
4.2 入口验证与准入 12
4.3 路线对比 13
4.4 最终路线：用系统公开接口完成直达 13
4.5 直达失败时怎么处理 14
第 5 章 核心模块实现 14
5.1 能力库与统一目录 14
5.2 安装探测与最近添加 14
5.3 推荐排序 15
5.4 桌面卡片 15
5.5 搜索：按字母浏览与定位 16
5.6 分类页与配置 17
5.7 直达启动：从能力条目到一次 Want 18
5.8 刷新编排 18
第 6 章 工程质量与验证 19
6.1 纯逻辑内核与自动化测试 19
6.2 真机验证 19

6.3 性能与稳定性 20
6.4 隐私与安全 20
第 7 章 创新点与比较优势 20
7.1 四个创新点 20
7.2 与同类方案的对比 20
7.3 社会与经济效益 21
第 8 章 总结与展望 21
8.1 总结 21
8.2 展望 21
附录 A 能力库构成 22
附录 B 术语表 22
参考文献 22

第 1 章 背景与意义

1.1 移动应用生态的“功能孤岛”

移动互联网经过十余年发展，已经形成了覆盖社交、支付、出行、购物、办公、生活服务、健康运动、娱乐等完整场景的应用生态。以本项目真实扫描的设备为例：一台普通手机上安装了多款与能力库相关的 App，能够提取出大量可被直接打开的功能入口。它们分布在多个分类中，其中大部分入口可以通过 Scheme 或 App Linking 直达目标页面，部分通用需求通过系统意图由用户选择目标应用。

但对用户而言，App 只是容器，真正被需要的是容器里的功能。付款要打开支付 App，路线规划要打开地图 App，打开文档要打开办公 App，查快递要打开购物或物流 App。同一类需求往往散落在多个 App 中，用户必须先做一次“选 App”的决定，再做一次“找入口”的决定。App 越多，后一个决定越难。

更隐蔽的问题在于，这些能力对系统是不可见的。系统知道用户安装了哪些 App，却不知道每个 App 能直接打开哪些页面；桌面只能摆放 App 图标，不能摆放“付款”“路线规划”“打开文档”这样的功能。能力被封装在各自的应用内，形成一个个“功能孤岛”。

1.2 用户侧：找功能比找 App 更难

我们把一次典型使用拆成六个步骤：想完成一件事、解锁手机、在桌面找 App、打开 App、在 App 里找入口、到达功能页。六步之中，只有第一步和最后一步与“事情本身”有关，中间四步都是与工具搏斗。对于高频功能，用户尚能靠肌肉记忆缩短路径；对于低频但重要的功能——查航班、开发票、挂号、查快递——每次都要重新寻找。

三类人群的痛点尤为明显。第一类是效率敏感型用户，他们每天多次使用支付、出行、文档、会议等功能，重复的寻找成本被高频放大。第二类是生活服务型用户，他们需要的功能分散在多个 App，且入口名称不统一，难以形成稳定记忆。第三类是长辈与数字弱势群体，他们不熟悉 App 品牌与层级结构，面对满屏图标常常无从下手。

此外，App 版本更新会调整菜单结构，用户刚记住的入口可能在下一次更新后消失。入口的不稳定性进一步提高了学习成本，也让“把常用功能固定在桌面上”成为一种刚需。

1.3 平台侧：HarmonyOS 卡片带来的机会

HarmonyOS NEXT 的 Stage 模型与 Form Kit 为上述问题提供了新的解法。桌面卡片可以常驻在桌面，不需要打开任何 App 就能展示信息与入口；卡片通过 postCardAction 可以直接触发路由

或拉起目标页面；App Linking、Scheme 与 Shortcut 为跨应用跳转提供了公开、稳定的协议；bundleManager.canOpenLink 则让应用可以在本地探测某条路由是否可被系统打开。

这意味着，“功能级入口”第一次具备了产品化的土壤：能力可以被发现、被验证、被组织、被推荐，并以卡片的形式直接呈现在用户最高频的桌面上。星门正是围绕这一机会设计的。

1.4 项目意义

对用户，星门把“找 App”变成“找功能”。常用功能抬手上桌，低频功能可以按拼音首字母或分类快速定位，长辈也能按文字找到服务，不必理解 App 的内部结构。

对生态，星门把 App 已经公开的入口转化为可发现、可复用、可组合的系统能力。它不要求 App 改造，不要求接入新的 SDK，也不抓取用户数据；它做的是把公开协议整理成一张可维护的能力目录，让“藏在三级页面里的功能”重新被看见。

对平台，星门验证了“卡片即入口”的可行性。桌面卡片不只是信息展示组件，也可以成为能力呈现的载体：推荐负责“此刻最可能用”，最近添加负责“新装 App 的能力”，分类负责“按场景浏览”。这为 HarmonyOS 卡片生态提供了一种可复制的产品范式。

1.5 项目目标与边界

项目目标可以概括为四句话：聚合多款 App 的原子能力；让每一次点击都落到目标功能页；让推荐有依据、可解释、可复现；让工程实现可测试、可维护、可扩展。

项目边界同样明确。第一，只使用 App 公开声明的路由，不破解、不注入、不抓取私有接口。第二，只保留一条最可靠路由，不在运行时做多路由降级。第三，启动失败时明确报告“暂无可用直达入口”，不回退到 App 首页假装成功。第四，所有能力、用量、安装快照均保存在本机，不上云、不采集用户内容。

第 2 章 产品定义与使用场景

2.1 一句话定义

星门 StarGate 是一个面向 HarmonyOS NEXT 的应用原子能力智能聚合与直达平台。它把多个 App 中可被直接打开的“功能”，聚合进一张桌面卡片和一组轻量页面，让用户不用先打开 App，就能一步抵达功能。

“原子能力”是本项目的核心概念。它不是 App，也不是网页，而是一条“可被系统直接打开的目标页面 + 一条最可靠路由 + 一组来源、验证状态与分类信息”。一条原子能力至少包含功能名、所属 App、目标包名、分类、路由、来源与置信度七个要素。

2.2 目标用户

第一类是效率敏感型用户。他们每天高频使用支付、出行、文档、会议等能力，希望把常用功能固定在桌面，减少重复寻找。第二类是生活服务型用户。他们的需求跨越多个 App，希望按“我要做的事”而不是“我要打开的 App”来组织入口。第三类是长辈与数字弱势群体。他们更需要文字化、分类化、可搜索的入口，而不是记住图标位置。第四类是愿意尝试新交互的用户。他们希望把常用功能固定到桌面，用更短路径完成高频任务。

2.3 四个入口

星门提供四个互补的入口，覆盖“抬手就用、知道名字就搜、按场景逛、按习惯调整”四种使用方式。

- 桌面卡片：常驻桌面，包含“推荐”“最近添加”两个功能框与分类 chips。点图标直接打开对应功能页；点“搜索”进入搜索页；点“设置”进入设置页。
- 搜索：与推荐、最近添加、分类并列的查找入口。按拼音首字母分为 A－Z# 段，左侧大字母标题、右侧常驻索引条，支持单字母拼音搜索与一键跳转；输入即筛，取消有明确语义。
- 分类页：按购物、社交、支付、出行、生活、娱乐、健康运动、工具效率、其他等类目浏览该类目的全部能力，保留搜索框与展开文件夹式网格。
- 设置：调整分类顺序与卡片显示偏好；分类支持长按拖动，顺序持久化并同步到卡片。设置从卡片进入，不依赖独立 App 主界面。

2.4 典型场景

通勤场景：早上出门，用户在桌面卡片上直接点“路线规划”或“乘车码”，不必先打开地图或支付 App。出差前，点“按航班号查询”，系统意图会引导用户选择航班应用并直接进入查询页。

办公场景：用户需要打开昨天的文档、加入会议或提交审批。卡片推荐会根据当日与当月使用频次，把 WPS、会议、审批等能力排在前面；如果用户记得功能名，直接在搜索页输入首字母即可。

生活场景：查快递、缴水电费、挂号、买药，这些低频但重要的功能散落在不同 App。分类页把它们按“生活”聚合，卡片推荐与搜索提供两条互补的查找路径。

长辈场景：长辈不需要理解 App 品牌与层级，只需在卡片上认字，或在搜索页按拼音首字母找。文字标签、较大字号与清晰的分类，降低了数字门槛。

2.5 使用流程

首次使用时，用户在桌面长按进入编辑状态，选择“卡片”，把 StarGate 的桌面卡片添加到桌面。卡片会自动读取本机能力库与用量快照，展示推荐与最近添加。用户点击任意图标，应用构

造一次明确的 Want，直接拉起目标功能页；如果目标入口不可用，卡片或页面会给出“暂无可用直达入口”的明确提示，而不是停留在首页。

用户也可以从卡片进入搜索、分类页与设置页。在设置页中，各分类支持长按拖动排序，顺序会持久化并实时同步到卡片；卡片显示偏好也在这里调整。

2.6 设计原则

- 单一最可靠路由：每条能力只保留一条最可靠路由，运行时不做多路由遍历与降级。
- 失败即报告：入口不可用时明确返回失败，不回退 App 首页，不制造“成功”的假象。
- 入口可用性验证：只有真机点击后能落到目标功能页的入口，才进入正式目录。
- 本地优先：能力库、用量、安装快照全部保存在本机，不依赖服务端，不上传用户数据。
- 可解释、可复现：推荐公式公开、权重固定、同分按 id 排序；同样的输入必然得到同样的结果。

第 3 章 总体设计

3.1 设计目标

星门的设计目标不是“做一个功能最多的启动器”，而是“做一条可信、可维护、可扩展的能力呈现链路”。围绕这一目标，系统需要满足四个约束：入口与数据解耦，页面与系统 API 解耦，核心逻辑可在 Node 环境直接测试，能力目录可在不重新打包的情况下更新。

3.2 五层架构

系统采用入口层、编排层、能力层、数据层、系统层五层结构，层与层之间单向依赖。入口层只负责展示与交互，不直接读写数据；编排层负责把数据组装成页面或卡片需要的形状；能力层承载排序、探测、分组、搜索等纯逻辑；数据层负责能力、用量、安装与顺序的持久化；系统层封装 FormExtensionAbility、UIAbility、Preferences 与跨应用启动等平台能力。



图 3-1　星门五层架构：入口 / 编排 / 能力 / 数据 / 系统

这样的分层带来三个直接收益。第一，核心算法不依赖 UI 与 SDK，可以在 Node 环境中用真实 .ets 源码直接跑单测。第二，页面只消费编排层产出的数据，布局调整不会影响排序与探测逻辑。第三，系统 API 被收敛在系统层，未来适配不同设备形态时只需替换这一层。

表 3-1　五层架构职责

层级	主要模块	职责
入口层	桌面卡片、搜索、分类页、设置	展示与交互，不直接读写数据
编排层	CardBinding、WidgetCardData、RefreshOrchestrator、PageRouter、LaunchDispatcher	组装数据、编排刷新、启动适配
能力层	RankingEngine、RecentAdditions、SearchFilter / PinyinUtil、LetterSections、CategoryOrder	纯逻辑能力，不依赖 UI 与 SDK
数据层	CapabilityRepository、EffectiveCatalog、InstallSnapshot、UsageStore、CategoryOrderStore、DiscoveryCandidateCatalog	能力、用量、安装与顺序的持久化
系统层	FormExtensionAbility、UIAbility、Preferences、bundleManager.canOpenLink、startAbility	系统生命周期与跨应用启动

3.3 能力从发现到呈现

能力链路分为三段：发现与生产、验证与准入、呈现与回流。发现段从能力库种子、应用模板、安装探测与候选目录中收集入口；准入段通过 EffectiveCatalog 做安装状态过滤，只把真机可打开的 L0 入口写入正式目录；呈现段由 RankingEngine、RecentAdditions、SearchFilter、

LetterSections 与 CategoryOrder 共同决定卡片和页面里出现哪些能力、怎么排序、怎么分组；回流段把点击用量、安装变化与分类顺序写回本机存储，驱动下一次推荐。

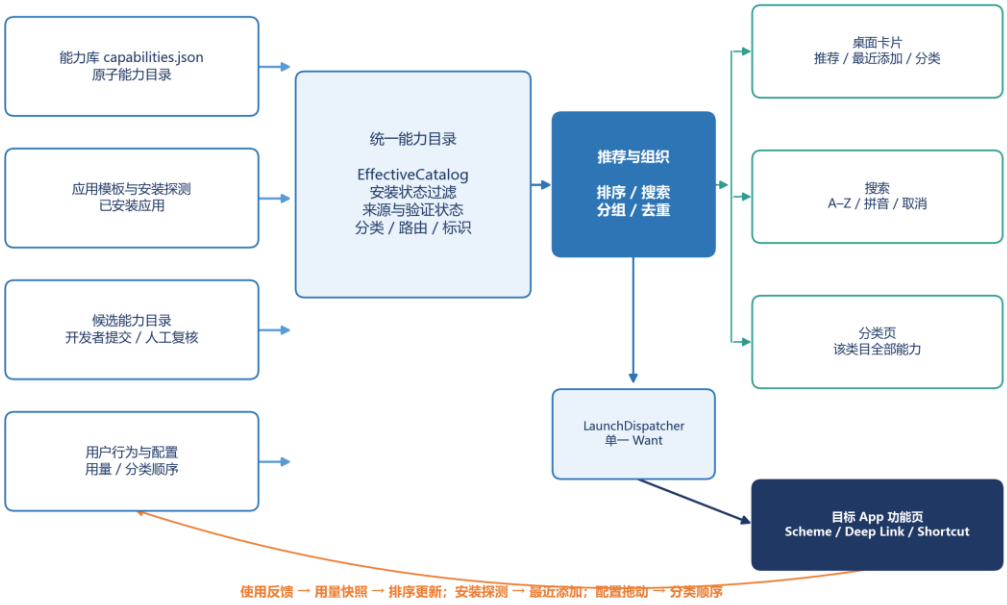


图 3-2　能力发现、准入与呈现

以一次点击为例：用户在卡片上点“路线规划”，LaunchDispatcher 根据该能力的单条路由构造 Want，调用系统 startAbility；成功后 UsageStore 记录一次使用，RankingEngine 在下一次刷新时提高该能力的分数；如果启动失败，CardFeedback 保留失败信息并在卡片上显示“暂无可用直达入口”。整个过程没有服务端参与，也不产生网络请求。

3.4 关键数据结构

系统围绕五个核心结构组织数据。AtomicFunction 描述一条原子能力；Route 描述一条最可靠路由；CardBindingData 描述一张卡片要渲染的全部内容；UsageSnapshot 描述用量；InstallSnapshot 描述安装与首次见到时间。所有结构均为显式接口或类，满足 ArkTS 的静态类型约束。

表 3-2　关键数据结构

结构	关键字段	作用
AtomicFunction	id、appName、bundleName、label、category、hint、route、source、confidence	一条可被呈现的原子能力
Route	kind、uri、abilityName、moduleName、parameters	单条最可靠路由，支持 deeplink / applinking / shortcut-param / want / vertical
CardBindingData	frames、categories、greeting、failure	卡片渲染数据：推荐、最近添加、分类与失败提示

结构	关键字段	作用
UsageSnapshot	dayCount、monthCount、lastUsedAt、hourAffinity、recentSequence	用量快照，驱动推荐排序
InstallSnapshot	installedIds、streak、bundleFirstSeen、bundlesProbedAt、appVersionFingerprint	安装状态、首次见到时间与去抖指纹

3.5 模块清单

项目按职责划分为数据、推荐、表单、页面、启动五组模块。数据组负责能力库、目录过滤、安装探测、用量、分类顺序与发现候选；推荐组负责排序与最近添加；表单组负责卡片绑定、卡片渲染、刷新编排与主题；页面组负责搜索、分类与设置；启动组负责 Want 构造、垂直入口与失败归一化。模块之间通过显式接口通信，避免隐式全局状态。

第 4 章 路线探索与最终选择

4.1 四条路线

在确定最终方案之前，项目组系统对比了四条把用户送到目标功能页的路线。它们不是四个互斥的“创意”，而是四种不同层面的工程路径：A 依赖应用主动声明，B 依赖应用公开的链接协议，C 依赖系统意图匹配，D 依赖对界面的逐步操作。

- A 应用内快捷方式：用户长按 App 图标，系统展示 Shortcut 菜单，选择后进入指定功能页。优点是系统原生、体验稳定；缺点是覆盖范围取决于 App 是否主动提供快捷方式，且用户必须先找到 App 图标。
- B 链接路由：通过 Scheme、App Linking、二维码或卡片链接直接唤起目标页面。优点是覆盖广、精度高、可跨应用组合；缺点是需要逐条验证路由是否可用，且裸 URI 可能被任意 App 认领。
- C 系统选择器：应用提出“我要查快递 / 我要导航”等能力请求，由系统根据 type 与 sceneType 选择合适的 App。优点是尊重系统意图、无需知道目标包名；缺点是用户每次都要做一次选择，直达精度较低。
- D UI 步骤自动化：预先录制或定制一串点击步骤，从桌面或 App 首页开始，依次点按菜单、标签与列表项，自动走到目标功能页。优点是直观、不依赖 App 公开路由，理论上任何可见功能都能到达；缺点是需要逐步维护 UI 脚本，速度慢、易受页面改版与加载时序影响，用户侧看到的是模拟操作而不是系统直达。

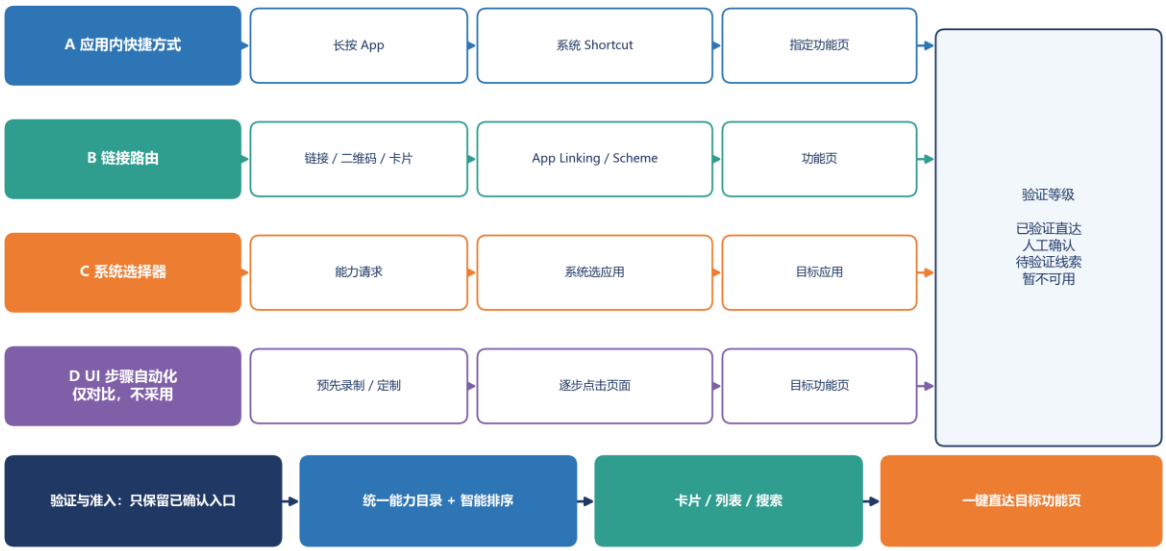


图 4-1　四条路线对比

4.2 入口验证与准入

不同来源的入口质量并不相同。项目组在工程侧按验证程度分级：已验证直达、人工确认、待验证线索与暂不可用。只有“已验证直达”和“人工确认”的入口可以进入正式目录；其余入口只留在候选目录，等待进一步验证。这套分级是内部质量控制手段，不进入用户界面。

表 4-1　入口验证等级与准入规则

验证等级	来源	含义	正式目录
已验证直达	系统 Shortcut、Link 特性、公开 Scheme	真机点击后可落到目标页面	是
人工确认	人工复核并录入	入口稳定、可复现	是
待验证线索	声明扫描或人工收集	已发现入口，尚未完成真机确认	否，留在候选目录
暂不可用	安装探测失败或目标未安装	当前设备上无法打开	否

4.3 路线对比

从覆盖范围、直达精度、用户心智、工程成本与可维护性五个维度对比，四条路线各有取舍。A 路线体验最好但覆盖最窄；B 路线覆盖与精度最均衡；C 路线尊重系统意图但牺牲了直达性；D 路线最直观，但每一步都依赖预先录制的界面操作，速度慢且容易被改版打断，最终不采用。

表 4-2 四条路线对比

维度	A 应用内快捷方式	B 链接路由	C 系统选择器	D UI 步骤自动化
覆盖范围	少，需 App 主动提供	广，公开 Scheme / App Linking	中，受系统意图限制	取决于脚本，可覆盖可见页面
直达精度	高	高	低，由系统选 App	能到页面，但依赖界面稳定
用户心智	长按才发现	链接 / 卡片自然	每次都要选 App	等待自动点击，链路长
工程成本	低	中	低	高，逐页录制与维护
可维护性	依赖 App 更新	中	高	低，页面改版即失效
项目中的用法	补充入口	主力路线	系统意图兜底	仅作对比，不采用

4.4 最终路线：用系统公开接口完成直达

最终产品不依赖私有通道，也不在运行时反复试探路由；它只使用 HarmonyOS 的公开接口：App 在清单里声明自己能响应什么，系统负责解析 Want 并找到目标 Ability，星门负责把一条经过验证的能力记录转换成一次明确的启动请求。

App 侧：在 module.json5 的 abilities[].skills[] 中声明 actions 与 uris，包括 scheme、host 与 path；提供快捷方式的应用在 shortcuts 中声明入口；使用 App Linking 的应用声明 HTTPS 域名。系统因此知道“这个 App 能接住哪些入口”，这些公开声明也是能力库的主要来源。

系统侧：Want 是跨应用启动的标准描述；bundleManager.canOpenLink(uri) 用于判断一条链接能否被系统打开；UIAbilityContext.startAbility(want) 按 bundleName 与 uri 找到目标 Ability 并拉起；对于“查快递”“导航”这类没有固定包名的需求，UIAbilityContext.startAbilityByType(type, wantParam) 会把能力类型交给系统匹配应用。

星门侧：能力库为每条能力保存一条最可靠路由；点击时只构造一个显式 Want，带上 bundleName 与 uri（或 abilityName、moduleName 与 parameters），调用 startAbility。启动失败就返回 NO_AVAILABLE_ENTRY，界面显示“暂无可用直达入口”，不回退 App 首页，也不尝试第二条路由。

以“高德地图 • 路线规划”为例。能力库记录 bundleName 为 com.amap.hmapp、uri 为 systemnav://routeplan；高德能够响应这个 URI。用户点击后，星门先确认链接可被系统打开，再构造一个显式 Want，交给 startAbility；系统在指定包内找到注册了该 URI 的 Ability，把请求

交给高德，高德直接打开路线规划页。整个过程不注入 UI、不模拟点击，只使用系统公开的启动协议。

```
const want: Want = {
  bundleName: "com.amap.hmapp",
  uri: "systemnav://routeplan"
};
await context.startAbility(want);
```

4.5 直达失败时怎么处理

如果目标 App 未安装、版本变化导致 URI 失效，或系统拒绝启动，星门不会静默回退到 App 首页，也不会连续尝试多条路由。界面会明确显示“暂无可用直达入口”，并把失败反馈留在卡片上，等待下一次安装探测或能力库更新。这样用户看到的每个“成功”都对应真实的目标页面，能力目录也能根据失败记录持续校正。

第 5 章 核心模块实现

5.1 能力库与统一目录

CapabilityRepository 是能力库的唯一入口，支持三种加载方式：外部可替换配置文件、HAP 内 rawfile 种子、直接注入的内存条目。开发期通过 hdc push 替换外部 JSON 即可更新能力库，不重新打包 HAP；生产环境回退到 rawfile 种子。加载器内置 UTF-8 解码器，逐字节处理 1 至 4 字节序列，避免中文 App 名被错误解码。

解析阶段执行严格校验：条目必须包含 id、appName、bundleName、route、category，route.kind 必须属于 deeplink、applinking、shortcut-param、want、vertical 之一，category 必须属于预定义的合法类目，confidence 必须在 0 到 1 之间。非法条目被丢弃，空文本或损坏文件返回空数组，保证空态不崩溃。

EffectiveCatalog 负责把“能力库”变成“当前可用的目录”：它根据安装状态过滤掉未安装 App 的能力，保留 L0 已验证入口，并把模板类条目排除在卡片之外。最终得到的目录只保留当前设备上已安装、且通过准入的能力，并按分类组织；具体规模随设备安装情况变化。

5.2 安装探测与最近添加

“最近添加”的定义是：用户最近安装或新检测到的 App 中，能力库能够覆盖到的功能。它替换了早期“最近打开”的设计，因为最近打开只反映旧习惯，不能帮助用户发现新能力。

实现上，InstalledProbe 使用 bundleManager.canOpenLink 对本机能力库中的路由进行探测，得到当前已安装的 bundle 集合；RecentAdditions.mergeSeen 负责维护“首次见到”时间表：首

次运行时把已有 App 全部记为 0，表示基线；之后新出现的 bundle 记为当前时间；本次未检测到的 bundle 从表中移除，卸载后重装会重新记录。

选择阶段由 RecentAdditions.select 完成：先按首次见到时间降序，再按能力库中的位置降序；同一个 App 只占一格，系统意图入口（vertical）不参与；如果没有任何新 App，则用能力库尾部的最新条目做默认填充。这样，新装 App 会自动排到前面并替换默认填充，用户不需要任何配置。

5.3 推荐排序

推荐排序由 RankingEngine 实现，采用加权公式：

$$score = 0.30 \cdot 当日使用 + 0.20 \cdot 当月使用 + 0.18 \cdot 时间衰减 + 0.12 \cdot 时段偏好 + 0.08 \cdot 场景亲和 + 0.07 \cdot 最近序列 + 0.05 \cdot 置信度$$

当日使用与当月使用按候选集最大值归一化；时间衰减使用 $\tau = 36$ 小时的指数衰减，让“昨天的使用”明显重于“上个月的使用”；时段偏好来自历史小时分布，冷启动时改用按当前小时给出的类目先验——上午偏出行、支付、工具效率，晚间偏购物、娱乐、生活；场景亲和首版保留接口、常量置 0；最近序列命中给 1；置信度来自能力库。

排序后执行类目打散：同一 category 最多 2 条，避免整屏都是购物或社交；候选不足时返回实际条数，绝不补位；同分按 id 字典序排序，保证结果可复现。

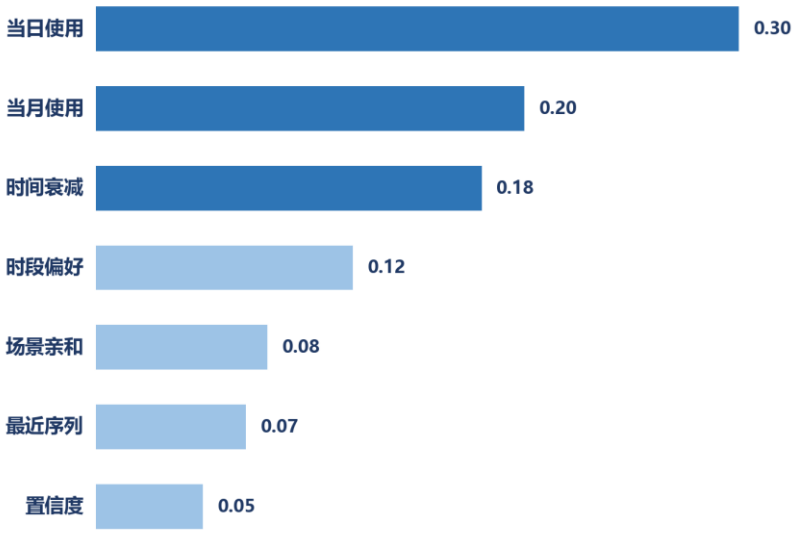


图 5-1　推荐排序的权重分布

5.4 桌面卡片

桌面卡片由 FormExtensionAbility 驱动。onAddForm 与 onUpdateForm 是同步回调，无法 await，因此卡片侧使用 CapabilityRepository.fromContextSync 同步读取能力库，再由 RefreshOrchestrator.resolve 组装 CardBindingData。卡片包含两个功能框：“推荐”展示排序

后的前 4 条能力，“最近添加”展示新装 App 的能力；下方是分类 chips，点击可进入对应分类页；右上角的搜索入口进入搜索页。

每个功能图标下方都有文字标签，最多两行、超出省略；图标与标签作为一个整体可点击。点图标时通过 postCardAction 触发 router，进入统一启动流程。卡片不展示“最近打开”与“收藏”，避免信息重复；失败时在卡片底部显示明确提示。



图 5-2　桌面卡片：推荐、最近添加、分类与搜索入口

5.5 搜索：按字母浏览与定位

搜索页复刻了 iOS“App 资源库”的信息结构：顶部是胶囊搜索框与“取消”，内容按拼音首字母分为 A－Z# 段，每段左侧是大写字母标题，行内是 46vp 圆角图标、功能名、App 名与细分隔线，右侧常驻 AlphabetIndexer 索引条。点击索引字母可直接滚动到对应分段。

搜索采用“一次加载、纯内存过滤”的策略：进入页面时读取全部能力并缓存，输入时只对缓存执行 SearchFilter 与 PinyinUtil，不再重复探测或读取文件。单字母搜索即可命中拼音首字母匹配的功能，例如输入 a 会返回“按航班号查询”“按起降地查询”。取消按钮遵循 iOS 语义：有输入时先清空，无输入时返回上一页。

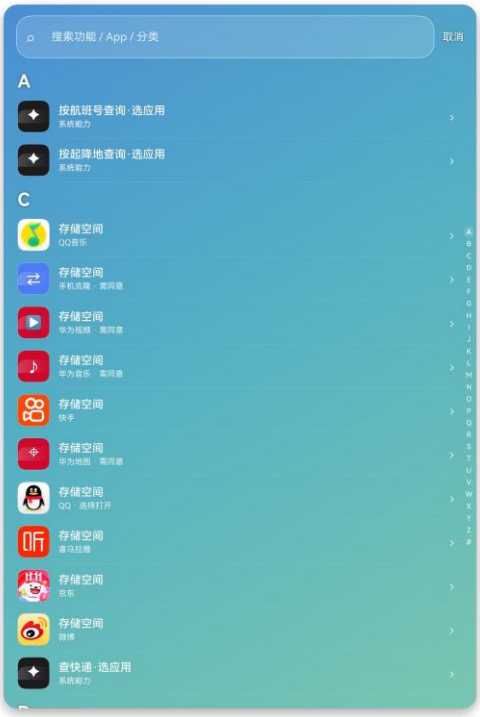


图 5-3　搜索页：A - Z# 分段与右侧索引



图 5-4　单字母拼音搜索

5.6 分类页与配置

分类页保留“展开文件夹”式网格：顶部是分类名与搜索框，内容区是一个玻璃质感的外层框，内部按每行 4 个排列功能图标。每个图标带功能名与 App 名，点击直接打开对应功能。页面在数据不足时从顶部开始排列，短内容不会被 Scroll 垂直居中。

设置页负责个性化。分类以 List 呈现，开启 editMode 后支持长按拖动；拖动开始时生成自定义预览，放下时由 CategoryOrder.move 计算新顺序，写入 CategoryOrderStore，并实时同步到卡片。能力库路径在设置页底部展示，开发期可直接替换外部 JSON。

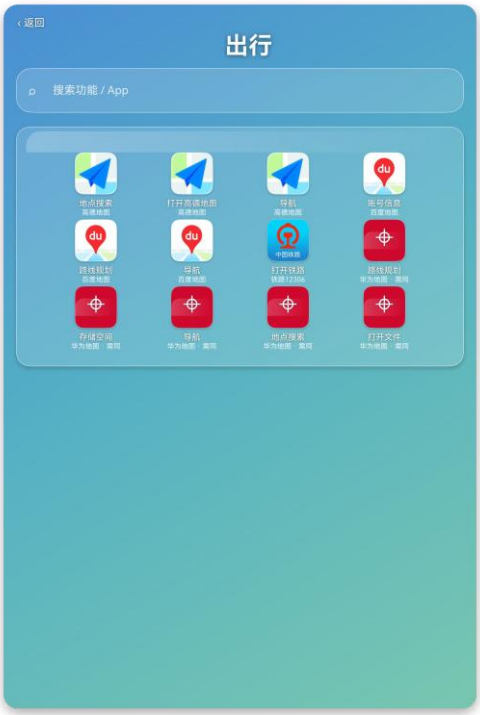


图 5-5　分类页：按场景浏览该类目全部能力

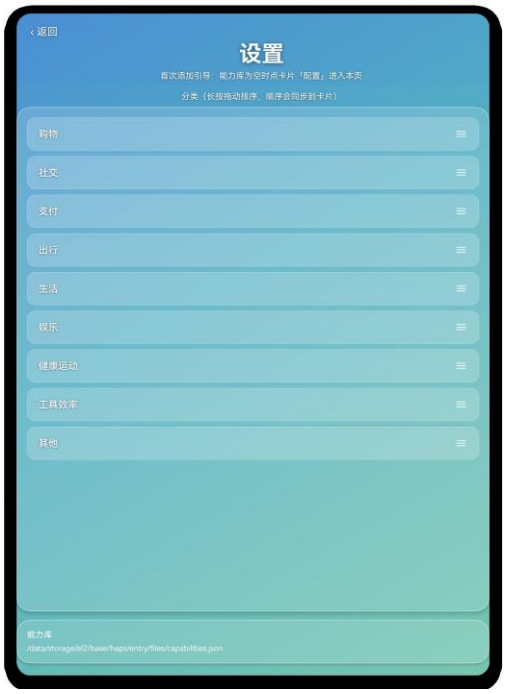


图 5-6　设置页：分类长按拖动排序

5.7 直达启动：从能力条目到一次 Want

LaunchDispatcher 是唯一的启动入口。它只读取 fn.route 这一条最可靠路由，按 kind 构造 Want：deeplink 与 applinking 使用 uri 与 parameters；shortcut-param 使用 abilityName、moduleName 与 parameters；want 使用 uri、action 与可选 ability；所有分支都显式带上 bundleName，避免裸 URI 被其他 App 认领。

vertical 是唯一例外：它不构造 Want，而是把 verticalType 与参数交给 startAbilityByType，由系统按类型匹配目标应用。启动成功后记录一次用量；失败则返回 ok=false 与 reason=NO_AVAILABLE_ENTRY，由界面归一化为“暂无可用直达入口”。运行时不尝试第二条路由，也不回退 App 首页。

安装探测阶段使用 bundleManager.canOpenLink 判断一条 URI 是否能被系统打开，把未安装或已失效的入口挡在目录之外；真正点击时以 startAbility 的返回结果为准。这样“目录里看得到”和“点下去打得开”使用同一套系统能力，不依赖界面自动化或私有接口。

5.8 刷新编排

RefreshOrchestrator 负责把能力库、模板、安装探测、用量、失败反馈与主题状态组装成一次刷新结果。它返回不可变快照，绝不修改上一轮状态；安装 diff 采用双次确认与版本指纹去抖，避免 App 安装过程中的瞬时变化导致卡片抖动；首次开启路由探测时建立“最近添加”基线，后续只把新出现的 bundle 记为新增。

第 6 章 工程质量与验证

6.1 纯逻辑内核与自动化测试

项目把排序、探测、去重、分组、搜索、路由构造等核心逻辑放在不依赖 UI 与 SDK 的纯 TypeScript 模块中，通过自研 Node 测试环境直接加载真实 .ets 源码运行。测试加载器提供 @ohos/hypium 与最小化的 SDK shim，使同一份源码既能在 ArkTS 编译器中构建，也能在 Node 中执行。

测试覆盖排序与推荐、安装探测、目录过滤、UTF-8 解码、搜索与分组、Want 构造、失败语义、卡片编排、页面静态约束等域。每次改动后先跑逻辑测试，再上真机验证；纯逻辑测试可在 30 秒内跑完。

表 6-1 自动化测试矩阵

测试域	代表用例	结果
排序与推荐	加权公式、类目打散、冷启动先验、同分确定性	通过
安装探测	首次基线、新 App 识别、卸载重装	通过
目录与过滤	UTF-8 解码、非法条目校验、安装状态过滤	通过
搜索与分组	拼音首字母、单字母搜索、A–Z# 分组	通过
启动与失败	四种 Want 构造、vertical 分支、失败即报告	通过
卡片编排	推荐 / 最近添加 / 分类、失败反馈、版本指纹	通过
页面静态约束	分段列表、索引、拖动排序、搜索栏内缩	通过

6.2 真机验证

逻辑测试通过后，项目在 HarmonyOS NEXT 真机上完成端到端验证。构建使用 DevEco Studio 自带的 hvigor 与 JBR 21，产出已签名 HAP；通过 hdc install -r 安装后，使用 uitest uiInput 驱动点击、长按、输入与拖动，并用 snapshot_display 截图留档。

表 6-2 真机验证记录

验证场景	操作	结果
搜索	查看 A–Z# 分段、点索引 C、输入 a、点取消	通过
分类页	从卡片分类 chips 进入、确认每行 4 个图标与文字标签	通过
设置	长按拖动“支付”到第 3 行、关闭重开确认持久化	通过
桌面卡片	确认推荐 / 最近添加 / 分类、图标文字完整、无重叠	通过
构建安装	hvigor 签名构建、hdc install -r、启动卡片与页面	通过

6.3 性能与稳定性

性能方面，搜索页只在进入时读取一次能力库，输入时只做内存过滤，避免每按一个键就跨 bundle 探测；卡片侧使用同步读取，避免异步回调丢失；安装探测结果按轮次缓存，同一轮刷新只探测一次。

稳定性方面，能力库解析对非法条目做丢弃而不是抛异常；刷新编排使用不可变快照，失败反馈在下一轮刷新中保留；卡片与页面共用同一个启动器，避免出现“卡片能开、页面不能开”的不一致。

6.4 隐私与安全

星门只读取 App 的公开路由声明，不抓取联系人、位置、相册、账号等用户数据；能力库、用量、安装快照与分类顺序全部保存在本机 Preferences 或应用沙箱文件中，不上传云端；每次启动都带明确 bundleName，不使用裸 URI，不静默落到首页；能力库为每条能力保留来源与验证状态，失败时给出明确提示。项目使用“公开入口”，不采集“用户数据”。

第 7 章 创新点与比较优势

7.1 四个创新点

第一，功能粒度的能力目录。传统启动器组织的是 App 图标，星门组织的是原子功能；每条能力包含目标页面、路由、分类、来源与置信度，可以直接被搜索、推荐与呈现。

第二，入口验证与准入。项目没有把扫描到的所有链接直接塞进产品，而是按来源与验证程度分级：只有真机确认可落到目标页面、或经过人工复核的入口才进入正式目录，其余线索留在候选池。这让“覆盖广”与“可信”同时成立。

第三，主动呈现而非被动陈列。卡片同时承载推荐、最近添加与分类：推荐负责“此刻最可能用”，最近添加负责“新装 App 的能力”，分类负责“按场景浏览”。三种逻辑互补，覆盖不同心情。

第四，可解释、可复现的推荐。加权公式、固定权重、36 小时衰减、类目打散与同分排序全部公开且确定；每条能力保留来源与验证状态；测试覆盖排序与探测的关键分支。推荐结果不是黑盒。

7.2 与同类方案的对比

与传统启动器相比，星门的组织单位从 App 变成功能，支持直达目标页面、按加权公式推荐、保留来源与验证状态，并能感知新装 App；与系统全局搜索相比，星门不只搜索 App 与网页，而是搜索可直达的原子功能，并提供卡片常驻入口与可追溯的来源标注。

表 7-1 与同类方案的对比

能力	传统启动器	系统全局搜索	星门 StarGate
组织单位	App 图标	App 与网页	原子功能
直达能力	只能开首页	部分 deep link	单条最可靠路由
推荐依据	使用频率	搜索历史	加权公式 + 时段 + 类目打散
可解释性	无	无	推荐公式 + 来源标注
最近添加	无	无	安装探测自动上桌
覆盖范围	已安装 App	可搜索内容	多款 App • 多项能力

7.3 社会与经济效益

社会效益方面，星门降低了数字门槛。长辈不必理解 App 品牌与层级，按文字或拼音即可找到服务；低频重要功能不再因为“藏得深”而被遗忘；公开入口被重新组织，延长了 App 功能的使用寿命。

经济效益方面，星门为 App 提供了一条不改造、不接入 SDK 的“功能曝光”通道。对中小开发者，功能可以被更多用户看见；对平台，卡片成为新的入口位；对用户，操作路径缩短，时间成本下降。三方收益在同一个能力目录上达成一致。

第 8 章 总结与展望

8.1 总结

星门 StarGate 完成了从“能力发现”到“用户直达”的完整闭环：以公开 Scheme、App Linking 与 Shortcut 为来源，以真机可用性验证为准入，以统一能力目录为单一事实源，以桌面卡片、搜索、分类与设置为呈现入口，以统一 Want 直达目标功能页。能力库覆盖主流应用与多个生活分类，并可随安装与发现持续扩展；自动化测试与真机验证全部通过；签名 HAP 可直接安装。

项目最重要的不是“做了多少功能”，而是验证了一种方法：App 是容器，功能才是服务；把功能从容器里请出来，需要对入口的敬畏、对可用性的坚持和对用户体验的克制。

8.2 展望

下一阶段，能力库将继续扩充，覆盖更多 App 与系统意图；推荐将引入时间、地点、设备状态等场景信号，让“此刻最可能用”更准确；能力接入规范将逐步沉淀，向开发者开放，让更多公开入口进入目录。星门已经打开，功能直达的路径会越来越短。

附录 A 能力库构成

表 A-1 能力库构成与准入方式

维度	构成	说明
能力来源	公开 Scheme / App Linking / Shortcut	应用公开声明的入口
扩展来源	开发者提交 / 人工复核 / 真机确认	补充未公开但可验证的入口
能力分类	出行、购物、办公、生活服务等	按用户场景组织，可继续扩充
呈现入口	桌面卡片 / 搜索 / 分类 / 设置	同一份目录，多种使用方式
准入状态	已验证直达 / 待验证线索	验证通过才进入正式目录

附录 B 术语表

表 B-1 术语表

术语	含义
原子能力	一条可被系统直接打开的目标页面及其最可靠路由、分类、来源与验证状态
L0	真机点击后可落到目标功能页的已验证入口
Want	HarmonyOS 的跨应用启动描述，包含 bundleName、uri、abilityName 等字段
App Linking	基于 HTTPS 域名校验的应用间跳转协议
Deep Link	基于自定义 Scheme 的应用内页面跳转
vertical	通过 startAbilityByType 交给系统按类型匹配目标应用的入口
验证等级	按来源与验证程度划分的准入分级，仅用于工程侧质量控制
最近添加	最近安装或新检测到的 App 中，能力库能够覆盖到的功能

参考文献

[1] 华为开发者联盟. HarmonyOS NEXT 应用开发文档[EB/OL]. https://developer.huawei.com/.

[2] 华为开发者联盟. Form Kit 卡片开发指南[EB/OL]. https://developer.huawei.com/.

[3] 华为开发者联盟. App Linking 开发指南[EB/OL]. https://developer.huawei.com/.

[4] 华为开发者联盟. ArkTS 语言规范[EB/OL]. https://developer.huawei.com/.

[5] 软件学院 SMAT 项目组. StarGate 能力库 capabilities.json（2026-11 版）[Z]. 2026.

[6] 软件学院 SMAT 项目组. StarGate 测试报告与真机验证记录[R]. 2026.